Cahier des charges — ArtisanConnect

Plateforme marketplace pour l'artisanat marocain

Version : 1.0 · **Date de référence :** 8 novembre 2026 · **Statut :** document de cadrage pédagogique

1. Présentation générale

1.1 Projet, contexte et problématique

ArtisanConnect est une plateforme web qui met en relation des artisans marocains et des clients désirant découvrir et acheter des produits artisanaux. Le projet répond à la faible visibilité numérique de nombreux ateliers, à la difficulté de comparer une offre dispersée et au besoin d'un suivi structuré des demandes de commande.

Le dépôt existant contient une application nommée **Al Asel**, fondée sur Flutter et une API Express/Firebase, déjà orientée profils d'artisans, demandes et messagerie. Il ne réalise pas encore le périmètre marketplace complet décrit ici (catalogue de produits, panier, commande et stock). Le présent dossier est un cadrage cible : l'équipe doit décider avec l'encadrant si elle l'étend à l'existant ou le réalise comme module séparé.

1.2 Justification et objectifs

La plateforme valorise le savoir-faire local, facilite la découverte par catégorie et centralise le cycle catalogue → panier → commande. Objectifs généraux : livrer un MVP démontrable, sécurisé et responsive; pratiquer l'analyse, la conception UML, la planification et les tests.

Objectifs spécifiques : permettre au client de trouver un produit; permettre à l'artisan de gérer un catalogue et son stock; donner à l'administrateur les outils de modération; conserver une trace fiable des commandes.

1.3 Public cible et périmètre

Public cible : clients particuliers, artisans/ateliers marocains et administrateur de plateforme. Le MVP couvre l'authentification, le catalogue, la recherche, le panier, les commandes, le stock et l'administration. Hors périmètre : paiement réel, calcul automatique de livraison, multi-vendeurs dans une même commande, application native, traduction complète et intégration ERP.

2. Étude des besoins

2.1 Acteurs

2.2 Besoins fonctionnels

Le système doit gérer l'inscription/connexion et les rôles; exposer un catalogue public avec catégories, recherche et filtres; afficher une fiche produit; conserver un panier par client; créer une commande; afficher les historiques; permettre à l'artisan de gérer profil, produits, stock et statuts de commande; permettre à l'administrateur de modérer utilisateurs, artisans, produits, catégories et commandes.

2.3 Besoins non fonctionnels

2.4 Contraintes, hypothèses et limites

Stack recommandée : React/Vite + Node.js/Express + PostgreSQL/Prisma ou adaptation prudente de l'API Express/Firebase existante. Git/GitHub, Trello et Mermaid sont obligatoires pour le suivi pédagogique. L'équipe ne doit pas affirmer la conformité légale ou la mise en production sans validation humaine.

3. Spécifications fonctionnelles

3.1 Client

3.2 Artisan

3.3 Administrateur

4. Spécifications non fonctionnelles détaillées

Le serveur est l'autorité pour le prix, le stock et les autorisations. Chaque endpoint protégé vérifie le jeton, le rôle et, pour une ressource artisan, la propriété. Les entrées sont validées, les requêtes sensibles journalisées sans mots de passe, les erreurs ne révèlent pas de détails internes. Les images reçoivent une limite de type/taille et un nom non prédictible. Les migrations, variables d'environnement et sauvegardes sont documentées.

5. Règles métier

1. Un compte a exactement un rôle actif : client, artisan ou administrateur; seul l'administrateur attribue le dernier rôle.

2. Un produit est publié par un seul artisan vérifié; titre, catégorie, prix strictement positif et stock entier ≥ 0 sont requis.

3. Un panier ne contient que des produits actifs et disponibles. Pour le MVP, il ne peut contenir que les produits d'un seul artisan; ajouter un autre artisan invite à vider ou terminer le panier.

4. Le prix et le stock sont revérifiés lors de la commande. La création est transactionnelle : commande et lignes sont créées, le stock est décrémenté, sinon aucune modification n'est conservée.

5. Une commande appartient à un client et un artisan; elle ne peut jamais être supprimée. Les changements de statut sont autorisés uniquement suivant le flux défini en F-ART-04.

6. Le client ne peut pas voir les commandes d'un autre client; l'artisan ne voit que les siennes; l'administrateur voit tout.

7. Un produit déjà présent dans une commande est archivé/désactivé, jamais effacé physiquement, afin de préserver l'historique.

6. Modélisation UML

6.1 Cas d'utilisation

flowchart LR

C[Client] --> UC1((Créer un compte / se connecter))

C --> UC2((Parcourir, rechercher et filtrer))

C --> UC3((Consulter un produit))

C --> UC4((Gérer le panier))

C --> UC5((Passer et suivre une commande))

A[Artisan] --> UA1((Gérer son profil))

A --> UA2((Gérer produits et stocks))

A --> UA3((Consulter / traiter les commandes))

ADM[Administrateur] --> UAD1((Gérer utilisateurs et artisans))

ADM --> UAD2((Gérer produits, catégories et commandes))

ADM --> UAD3((Superviser la plateforme))

UC5 --> S[(Catalogue / commandes)]

UA2 --> S

UA3 --> S

6.2 Diagramme de classes

classDiagram

class Utilisateur {

+UUID id

+String nom

+String email

+String motDePasseHash

+Role role

+DateTime creeLe

}

class Artisan {

+String nomAtelier

+String description

+String ville

+String telephone

+StatutVerification statut

}

class Client

class Categorie { +UUID id +String nom +String slug }

class Produit {

+UUID id

+String titre

+Text description

+Decimal prix

+Int stock

+Boolean actif

}

class Panier { +UUID id +Decimal total }

class LignePanier { +Int quantite +Decimal prixUnitaire }

class Commande {

+UUID id

+StatutCommande statut

+Decimal total

+DateTime creeLe

}

class LigneCommande { +Int quantite +Decimal prixUnitaire }

Utilisateur <|-- Client

Utilisateur <|-- Artisan

Artisan "1" --> "*" Produit : publie

Categorie "1" --> "*" Produit : classe

Client "1" --> "0..1" Panier : possède

Panier "1" *-- "*" LignePanier

LignePanier "*" --> "1" Produit

Client "1" --> "*" Commande : passe

Commande "1" *-- "*" LigneCommande

LigneCommande "*" --> "1" Produit

Artisan "1" --> "*" Commande : reçoit

6.3 Diagrammes de séquence

Scénario S1 — passer une commande

sequenceDiagram

actor Client

participant Web as Application web

participant API as API

participant DB as Base de données

Client->>Web: Ajoute un produit au panier

Web->>API: POST /cart/items

API->>DB: Vérifie produit, stock et prix

DB-->>API: Produit disponible

API-->>Web: Panier mis à jour

Client->>Web: Confirme la commande

Web->>API: POST /orders

API->>DB: Transaction : crée commande, décrémente stock

DB-->>API: Commande créée

API-->>Web: Confirmation et référence

Scénario S2 — artisan met à jour un produit

sequenceDiagram

actor Artisan

participant Web as Tableau de bord

participant API as API

participant DB as Base de données

Artisan->>Web: Enregistre la modification

Web->>API: PATCH /products/:id

API->>API: Vérifie JWT, rôle et propriété

API->>DB: Met à jour produit

DB-->>API: Produit enregistré

API-->>Web: Produit actualisé

7. Architecture proposée

Architecture web trois couches : un front-end React responsive communique en HTTPS avec une API REST `/api/v1`; l'API Express applique authentification, validation et règles métier; PostgreSQL conserve les données relationnelles et un stockage objet conserve les médias. Cette architecture reste réalisable par des étudiants, teste les services sans navigateur et évite de placer des secrets dans le front-end.

flowchart LR

B[Navigateur React/Vite] -->|HTTPS / JSON| A[API Express]

A --> AU[Service Auth / RBAC]

A --> CA[Service Catalogue]

A --> CO[Service Commandes]

AU --> DB[(PostgreSQL)]

CA --> DB

CO --> DB

CA --> ST[Stockage images]

8. Planning prévisionnel

Le planning de huit semaines, les dépendances et les jalons sont détaillés dans `../02_Gestion_de_projet/02_Diagramme_Gantt.mmd` et `../02_Gestion_de_projet/03_Diagramme_PERT.md`. Les documents, la préparation de soutenance et certaines maquettes sont parallélisables; intégration et recette restent séquentielles après les noyaux front-end/back-end.

9. Critères de validation globaux

- Les scénarios F-CLI-01 à F-CLI-07, F-ART-01 à F-ART-04 et F-ADM-01 à F-ADM-04 sont démontrés avec données de test.

- Les permissions interdites retournent une réponse contrôlée et ne modifient aucune donnée.

- Une commande refuse une quantité supérieure au stock et conserve le prix validé au moment de la création.

- Le catalogue fonctionne sur mobile et bureau; les erreurs et états vides sont traités.

- Les tests automatisés essentiels passent, le README permet une installation reproductible et la recette signée est archivée.

10. Conclusion et perspectives

Le MVP privilégie un parcours fiable plutôt qu'une large couverture fonctionnelle. Après validation, les évolutions possibles sont paiement sécurisé via prestataire, livraison, avis vérifiés, messagerie, multi-vendeur, langues arabe/français et tableau de bord analytique.